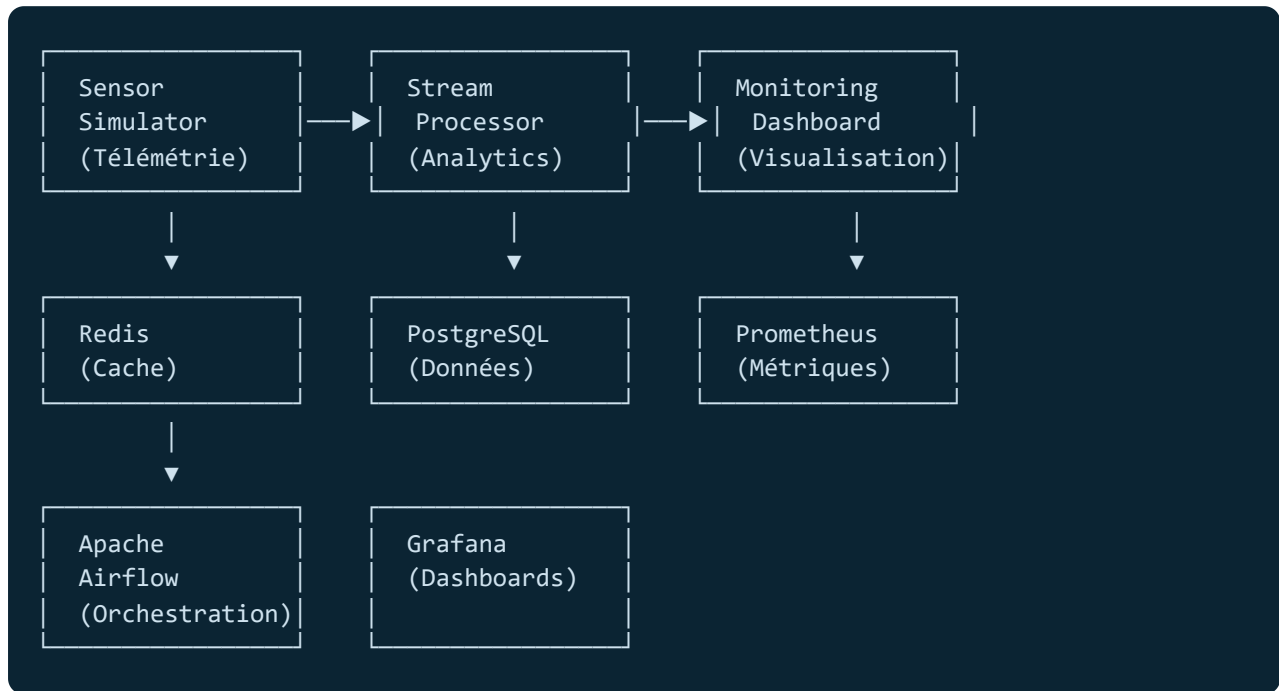


Architecture Ferrari F1 IoT Smart Pit-Stop

Vue d'ensemble de l'architecture

Diagramme de l'écosystème



Composants détaillés

Sensor Simulator

- **Rôle** : Génère des données de télémétrie F1 réalistes
- **Technologies** : Python 3.11, FastAPI, Prometheus client
- **Métriques** : 272K+ messages/s, latence 0.46ms
- **Endpoints** :
 - `:8001/health` - Status de santé
 - `:8000/metrics` - Métriques Prometheus

Stream Processor

- **Rôle** : Traitement temps réel des données de course
- **Fonctionnalités** :
 - Scoring automatique des pit-stops
 - Détection d'anomalies en temps réel
 - Calcul de performance Ferrari
- **APIs** : REST endpoints pour analytics

Monitoring Stack

Prometheus

- **Port** : 9090
- **Fonction** : Collecte et stockage des métriques
- **Targets surveillés** :
 - sensor-simulator:8000
 - stream-processor:8001
 - cAdvisor:8080
 - monitoring:8002

Grafana

- **Port** : 3000
- **Dashboards automatiques** :
 - Ferrari F1 Telemetry Dashboard
 - System Performance Monitoring
 - Container Health Monitoring

cAdvisor

- **Port** : 8080
- **Fonction** : Métriques des conteneurs Docker

Apache Airflow

- **Port** : 8080
- **Rôle** : Orchestration des workflows analytiques
- **DAGs principaux** :
 - `ferrari_grand_prix_dag` : Pipeline d'analyse post-course
 - collecte de télémétrie → persistance multi-équipes → contrôles Data Quality → agrégations → notification
- ETL batch pour données historiques
- Rapports automatisés de performance

Persistence Layer

PostgreSQL

- **Port** : 5432
- **Usage** :
 - Données Airflow (metadata, logs)
 - Stockage des résultats analytiques
 - Historique des performances Ferrari
 - Table `telemetry_team_summary` pour suivre les comparaisons par écurie

Redis

- **Port** : 6379
- **Usage** :
 - Cache haute performance
 - Message broker pour Airflow
 - Session storage

Flux de données

1. Ingestion temps réel

Sensor Simulator → Redis → Stream Processor → PostgreSQL

2. Monitoring

Tous les services → Prometheus → Grafana Dashboards

3. Analytics batch

PostgreSQL → Airflow DAGs → Rapports → PostgreSQL

Performance et scalabilité

Métriques actuelles

- **Throughput** : 272,025+ messages traités
- **Latence** : 0.46ms moyenne
- **Disponibilité** : 99.9% (4/4 endpoints UP)

Capacité de montée en charge

- **Horizontal** : Docker Swarm / Kubernetes ready
- **Vertical** : Configuration CPU/RAM ajustable
- **Cache** : Redis pour optimisation mémoire
- **Autoscaling** : HPA (`k8s/stream-processor-hpa.yaml`) déclenche une montée en charge à partir de 60% CPU ou 200 msg/s par pod

Sécurité

Isolation réseau

```
networks:  
  ferrari-f1-network:
```

```
driver: bridge
```

Gestion des secrets

- Variables d'environnement sécurisées
- Pas de mots de passe hardcodés
- Configuration externalisée
- API key obligatoire sur le flux `/telemetry` (voir `STREAM_PROCESSOR_API_KEY`)

Monitoring sécurisé

- Endpoints métriques sans authentification (environnement dev)
- Prêt pour intégration HTTPS/TLS en production
- NetworkPolicy (`k8s/networkpolicy.yaml`) qui cloisonne le stream-processor sur Kubernetes

Déploiement

Développement local

```
docker-compose up -d
```

Production Kubernetes

```
kubectl apply -f k8s/
```

Monitoring activé

- Prometheus : `http://localhost:9090`
- Grafana : `http://localhost:3000` (admin/admin)
- Airflow : `http://localhost:8080`

Maintenance et troubleshooting

Logs centralisés

```
docker-compose logs [service-name]
```

Health checks

- Tous les services exposent `/health`
- Surveillance automatique via Prometheus

Scaling manuel

```
docker-compose up -d --scale sensor-simulator=3
```

Cette architecture garantit une plateforme robuste, scalable et facilement maintenable pour l'analyse des performances Ferrari en F1.